

PROYECTO COMPUTACIÓN VISUAL 2025- I

Nombre : ClasifEye

Integrantes

- Gabriela Guzmán Rivera - Gabriela Guzman Rivera
- Juan Esteban Cardenas Huertas - Juan Esteban Cardenas Huertas
- Juan David Ardila Diaz - Juan David Ardila Diaz
- Manuel Eduardo Díaz Sabogal - Manuel Eduardo Diaz Sabogal
- Gabriela Gallegos Rubio - Gabriela Gallegos Rubio

Justificación

La capacidad de detectar rostros humanos en imágenes y clasificar automáticamente atributos como edad y género tiene aplicaciones prácticas significativas que incrementan cada día más, la necesidad de desarrollar estos sistemas, cómo se ve en la personalización de experiencias de usuario, la mejora de sistemas de vigilancia, el análisis demográfico en tiempo real y el desarrollo de interfaces hombre-máquina más intuitivas.

El propósito de este proyecto es poner en práctica los conceptos aprendidos en la materia de Computación visual para lograr mejor la comprensión de conceptos y a su vez desarrollar el ámbito investigativo en los estudiantes, complementado por el aprendizaje autónomo para fortalecer las habilidades técnicas en el manejo de bibliotecas de código abierto, la implementación de redes neuronales convolucionales (CNN) y la creación de aplicaciones web interactivas.

Además, promueve el trabajo colaborativo, la planificación estructurada y la resolución de problemas complejos, habilidades esenciales para el desarrollo profesional en el campo de la ingeniería y la tecnología. Al emplear técnicas como Transfer Learning y arquitecturas eficientes (por ejemplo, MobileNet), el proyecto garantiza un equilibrio entre precisión y viabilidad computacional, lo que lo hace escalable y accesible incluso en entornos con recursos limitados.

Implícitamente el proyecto aborda desafíos éticos y sociales al incluir un análisis explícito de sesgos en los resultados, lo que permite reflexionar sobre las limitaciones de los modelos de IA y fomentar un desarrollo tecnológico responsable.

Objetivo

- Desarrollar un sistema basado en IA que detecte rostros humanos en imágenes y clasifique automáticamente su género y determine un rango de edad.

Tecnologías principales

- Python, OpenCV, TensorFlow/Keras.

Conjuntos de datos

- UTKFace, FaceARG, o CelebA (según disponibilidad).

Objetivos smart para para 10 semanas

• Semana 1: Investigación y Fundamentos

Configuración del Entorno: Crear repositorio (Git), configurar entorno de desarrollo Python (virtualenv/conda), instalar bibliotecas base (NumPy, Pandas, Matplotlib).	Análisis Detallado de Datasets: Investigar a fondo UTKFace, FaceARG, CelebA: estructura, tamaño, formato de etiquetas, pros/contras específicos para el proyecto, proceso de obtención/acceso.	Investigación de Herramientas: Revisar arquitecturas CNN comunes para clasificación de imágenes (VGG, ResNet, MobileNet) y el concepto de Transfer Learning.	Boceto Inicial de Arquitectura: Dibujar un diagrama de flujo de alto nivel: Imagen -> Detector -> Preprocesador -> Clasificador(es) -> Resultados. Discutir si usar un modelo multi-tarea o modelos separados.
--	--	--	--

• Semana 2: Definiciones y Planificación Detallada

Selección Final de Dataset: Tomar una decisión informada sobre el dataset principal (probablemente UTKFace por conveniencia) y planificar la descarga/acceso. Considerar si se usará un dataset secundario para aumentar diversidad o validar.	Selección Final de Herramientas: Decidir sobre el detector facial y la estrategia de clasificación (ej. Transfer Learning con MobileNetV2 por eficiencia).	Definición Detallada de Arquitectura y Pipeline: Especificar tamaños de entrada/salida para cada módulo, formato de datos intermedios (ej. cara recortada y redimensionada a 224x224), y cómo se conectarán los componentes.	Asignación Detallada de Tareas: Confirmar responsables específicos para las fases de preprocesamiento y entrenamiento, asegurando claridad en las subtareas.
--	---	--	---

• Semana 3: Adquisición y Preprocesamiento Inicial

Descarga y Organización de Datos: Descargar el dataset elegido y organizarlo en una estructura de carpetas lógica (ej. train/validation/test).	Script de Carga y Exploración: Escribir scripts para cargar imágenes y sus etiquetas (edad, género). Realizar análisis exploratorio: distribución de clases (histogramas), visualización de ejemplos.	Implementación de Detección/Recorte Básico: Si se usa detección para preprocesar: implementar el detector elegido para extraer rostros. Si se usan bounding boxes del dataset: escribir script para recortar los rostros según las etiquetas. Redimensionar a tamaño fijo.	Limpieza Inicial de Datos: Identificar y manejar imágenes corruptas, etiquetas faltantes o inconsistentes encontradas durante la exploración.
---	--	---	--

• Semana 4: Preprocesamiento Avanzado y Preparación

Normalización de Imágenes: Implementar la normalización de píxeles (ej. escalar a [0, 1] o usar la normalización específica del modelo pre-entrenado elegido).	Implementación de Aumento de Datos (Data Augmentation): Escribir funciones para aplicar transformaciones (rotaciones leves, volteo horizontal, cambios de brillo/contraste) para aumentar la variabilidad del training set.	División Estratificada de Datos: Crear particiones de entrenamiento, validación y prueba, asegurando que la proporción de clases (especialmente las minoritarias) se mantenga en cada split.	Creación de Generadores/Pipelines de Datos: Implementar generadores (ej. ImageDataGenerator de Keras o tf.data.Dataset) para cargar datos eficientemente durante el entrenamiento, aplicando augmentation "on-the-fly".
---	--	---	--

- Semana 5: Construcción y Entrenamiento Inicial del Modelo**

Implementación de Arquitectura CNN: Codificar una red neuronal convolucional utilizando Transfer Learning (ej. MobileNetV2) y añadir capas personalizadas para las salidas de clasificación de género y clasificación de edad en rangos.	Codificación de Etiquetas: Transformar las etiquetas de edad en clases discretas por rangos y codificar el género como variables binarias, integrándolas directamente en el DataFrame utilizado para entrenamiento.	Eliminación de Keypoints: Adaptar el pipeline de entrada de datos para eliminar cualquier dependencia de los keypoints faciales, ajustando la arquitectura y los datos para trabajar solo con imágenes 224x224.	Prueba de Entrenamiento Inicial: Ejecutar una prueba de entrenamiento corta (smoke test) con un subconjunto reducido del dataset para validar la integración completa del pipeline de datos y modelo.
---	--	--	--

- Semana 6: Entrenamiento, Evaluación y Ajuste del Modelo**

Entrenamiento Completo del Modelo: Ejecutar el entrenamiento del modelo CNN en el conjunto de datos completo durante varias épocas (20-25), registrando pérdidas y métricas de validación por cada salida.	Evaluación del Modelo en Validación: Calcular métricas detalladas como accuracy para género y matriz de confusión para rangos de edad en el conjunto de validación.	Análisis de Errores: Identificar patrones de error en las predicciones (por ejemplo, confusión entre rangos de edad adyacentes) mediante la visualización de ejemplos mal clasificados.	Ajuste de Hiperparámetros: Modificar hiperparámetros clave (tasa de aprendizaje, batch size) en función de los resultados obtenidos, y decidir si es necesario un nuevo entrenamiento parcial, si es necesario.
---	--	--	--

- Semana 7: Integración con Detección Facial**

Desarrollo del Módulo de Detección: Implementar un detector facial (ej. MTCNN u OpenCV Haar) que localice y retorne las coordenadas de la cara más prominente en una imagen.	Extracción y Preparación de Rostros: Diseñar una función que recorte la región facial detectada, la redimensione a 224x224 y aplique la misma normalización utilizada en entrenamiento.	Manejo de Casos Especiales: Establecer un protocolo para tratar casos sin rostros detectados o con múltiples rostros (ej. elegir el más grande o mostrar mensaje de advertencia).	Validación del Módulo de Detección: Probar el módulo de detección y extracción con un conjunto variado de imágenes para garantizar su funcionalidad y robustez.
---	--	--	--

- Semana 8: Pipeline Completo y Clasificación**

Optimizar el dataset y pipeline de entrenamiento: Optimizar el dataset mediante la revisión y eliminación de datos inconsistentes o con etiquetas dudosas, y aplicar técnicas adicionales de aumento de datos para mejorar la generalización.	Reentrenar el modelo CNN: A partir de los ajustes recientes, incluyendo mejoras en el preprocesamiento, aumentos de datos y cambios en la arquitectura, reentrenar la CNN y si aplica, monitorear las curvas de aprendizaje y métricas de validación.	Validar mejoras a partir del reentrenamiento: Comparar los resultados con versiones anteriores, enfocándose en métricas por clase (accuracy, etc) y en la reducción de errores en rangos adyacentes de edad o género mal clasificado.	Pruebas End-to-End: Ejecutar pruebas completas del pipeline con diferentes imágenes para detectar posibles errores en la integración y asegurar su correcto funcionamiento.
--	--	--	--

- Semana 9: Evaluación de métricas y graficación**

Unificar todos los módulos desarrollados (detección, preprocesamiento, clasificación) en un pipeline funcional completo que tome una imagen de entrada y devuelva predicciones formateadas, asegurando robustez mediante pruebas end-to-end.	Ejecutar una evaluación exhaustiva del modelo actualizado, calculando métricas finales sobre el conjunto de prueba, incluyendo precisión, recall y F1-score para cada clase, además de la matriz de confusión.	Analizar el comportamiento del modelo en subgrupos específicos (género y rango de edad), identificando posibles sesgos o desigualdades de rendimiento, y proponiendo ajustes o limitaciones documentadas.	Generar visualizaciones clave del entrenamiento y evaluación, incluyendo curvas de pérdida y accuracy por época, métricas por clase y ejemplos visuales de errores frecuentes para soporte en la presentación y análisis.
---	---	--	--

- Semana 10: Pruebas Finales, Documentación y Entrega**

Completar la documentación técnica del proyecto, incluyendo el diseño del modelo, decisiones técnicas, problemas encontrados, análisis de errores, y resultados comparativos tras los reentrenamientos.	Diseñar y preparar la presentación final del proyecto, estructurada en: motivación, diseño, resultados, aprendizajes y limitaciones. Incluir demo técnica del pipeline con ejemplos reales.	Realizar pruebas de usabilidad con usuarios del entorno, recolectando retroalimentación sobre la claridad de resultados, interfaz (si aplica) y tiempo de respuesta, implementando pequeños ajustes si son factibles.	Consolidar el modelo final con hiperparámetros ajustados (como tasa de aprendizaje y batch size), y garantizar su reproducibilidad mediante scripts listos para ejecución y guía de uso en el README.
--	--	--	--